דף סיכום בחינה

מזהה סטודנט: N102223367

מזהה קורס: 20594   שם קורס: מערכות הפעלה

מספר מבחן: 0001

מספר שאלה בטופס בחינה זה	ניקוד מירבי	ציון	התשובה שסומנה	התשובה הנכונה	מיקום השאלה בטופס המקור	מיקום התשובה שסומנה בטופס המקור
1	5.00	5.00	2	2	1	2
2	5.00	5.00	3	3	2	3
3	5.00	0.00	1	3, 4	3	1
4	5.00	5.00	3	3	4	3
5	5.00	5.00	4	3, 4	5	4
6	5.00	5.00	4	4	6	4
7	23.00	23.00	-	-	7	-
8	17.00	15.00	-	-	8	-
9	15.00	14.00	-	-	9	-
10	5.00	5.00	-	-	10	-
11	5.00	5.00	-	-	11	-
12	5.00	5.00	-	-	12	-

כמות תשובות נכונות (עבור שאלות סגורות) : 5

ציון בחינה סופי : 92.00

מקרא צבעים

תשובה שגויה
שאלה שלא נענתה
שאלה מבוטלת

הבחינה הבדוקה בעמודים הבאים

האוניברסיטה
הפתוחה

דף תשובות

מספר גרסה	מספר תעודת זהות	מספר הקורס	תאריך בחינה
0001	328197462	20594	100324

0	●	●	●	○
1	○	○	○	●
2	○	○	○	○
3	○	○	○	○
4	○	○	○	○
5	○	○	○	○
6	○	○	○	○
7	○	○	○	○
8	○	○	○	○
9	○	○	○	○

הוראות למילוי דף התשובות

1. נא למלא תשובה אחת בלבד על ידי השחרת העיגול - ●
2. את ההשחרה יש לבצע בעט שחור או כחול בלבד!
3. השימוש בטיפקס או מרקרים אסור!
4. כדי לתקן סימן, יש לסמן X על התשובה הלא נכונה - ⊗
ולהשחיר את התשובה הנכונה - ●
5. כדי לחזור לסימון המקורי, יש לסמן X על התשובה הלא רצויה,
ולהשחיר מחדש את סימן ה-⊗ באופן שיכסה המחיקה - ●

שאלה / תשובה	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
א	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	●	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	●	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלה / תשובה	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלה / תשובה	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלון 511 92.52.5 M2

3|2|8|1|9|7|4|6|2

מספר תעודת זהות
יש לרשום את כל תשע הספרות

ש מ N102223367

ת.ז: 328197462 מספר סידורי: **19**

האוניברסיטה הפתוחה

ל' באדר א' תשפ"ד

מס' שאלון - 511

10

במרץ 2024

סמסטר 2024א

20594 / 4

מס' מועד 92

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **10** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלק א' (שאלות אמריקאיות) עליכם לבחור בכל פעם
בתשובה יחידה הנכונה ביותר מבין התשובות.
את התשובות בחלק א' יש לסמן בטופס המחשב המצורף לשאלון.

ג. בחלקים ב' ו ג' מופיעות שאלות פתוחות.
ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ.
אין חובה להשתמש בכל השורות המוקצות לצורך התשובות,
אך אין לחרוג מהמקום המוקצה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.
אין להכניס חומר דיגיטלי!

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

שאלון 511 92.52.5 M2

הנחיות למבחן

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלק א׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות. **את התשובות בחלק א׳ יש לסמן בטופס המחשב המצורף לשאלון.**
2. בחלקים ב׳ ו ג׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות**, **בכתב קריא ובקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (30 נקודות)

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולסמן את התשובה **בטופס המחשב המצורף לשאלון.**

שאלה 1

בגרף המצבים הבסיסי של תהליך במערכת Linux, תהליך שמסיים עובר למצב zombie . הסיבה היא:

א. לשמור על מבנה הנתונים למקרה שנצטרך אותו שוב כשתהליך יבצע fork

[✓ ב.] לשמור מידע על התהליך שסיים עד שהאבא שלו יבצע wait

ג. לשמור מידע על התהליך שסיים עד שהמערכת תאסוף מידע על הפעילות של משתמשים שונים.

ד. זה מצב זמני כדי לחכות עד שנגמרות כל פעולות ה-I/O הכרוכות בסיום התהליך

שאלה 2

איזו תכונה חשובה **הכרחית ומספיקה** למעבד CPU בשביל שיהיה ניתן לממש עליו את ביצועי וירטאליזציה יעילה בשיטת Trap-and-Simulate?

א. המעבד יכול לפעול גם ב kernel mode וגם ב user mode

✗ ב. כל הפקודות המיוחסות(privileged instructions , הגורמות ל TRAP) מהוות תת-קבוצה של הפקודות הרגישות(sensitive instructions) במערכת

[✓ ג.] כל הפקודות הרגישות(sensitive instructions) במערכת מהוות תת-קבוצה של פקודות מיוחסות(privileged instructions , הגורמות ל TRAP)

ד. יכולת לבצע הפקודות הרגישות(sensitive instructions) קיימת רק ב kernel mode וב user mode המעבד פשוט לא מבצע אותן(למעשה שווה ל NOOP)

שאלה 3

זמן הריצה של התוכנית החדשה הוא 2 ms. מהי שיטת התזמון שתוביל לזמן התגובה הנמוך ביותר של התכנית?

הניחו שבמערכת רצות כבר 3 תכניות אחרות והן תרוצנה עוד זמן רב. התכניות חדשות מוכנסות לראש התור ורצות מיד.

[✓ א.] round robin עם quantum של 1 ms   זמן תגובה 3q 1ms

ב. round robin עם quantum של 10 ms   זמן תגובה 3q 10ms

ג. shortest remaining time first   מע׳ אצלנו! אנחנו רוצים אינטראקטיבית!

ד. זמן התגובה יהיה זהה בכל המקרים כיוון שהתכנית החדשה תהיה בראש התור ותרוץ ראשונה.

שאלה 4

מבין האלגוריתמים הבאים להחלפת דפים במערכות זיכרון וירטואלי, איזה צפוי להקטין את מספר ה-page faults למינימום? הנח כי כל האלגוריתמים אכן ניתנים למימוש.

א. LRU

ב. אלגוריתם השעון

[✓ ג.] אלגוריתם הבוחר להחליף את הדף שלא נצטרך אותו הכי הרבה זמן בעתיד

ד. אלגוריתם הבוחר להחליף דף שאינו חלק מה-working set, כלומר שלא השתמשנו בו ב-Δ הפקודות האחרונות, עבור Δ מתאים

שאלה 5

באלגוריתם הבנקאי:

א. מצב בטוח הוא מצב שבו אין deadlock

ב. מצב בטוח הוא מצב שבו אין שום סכנה ל-deadlock בעתיד

ג. מצב בטוח הוא מצב שממנו אפשר להגיע ל-deadlock, אבל אפשר יהיה להתחמק מכך על ידי חסימת תהליכים מסויימים במקרה הצורך.

[✓ ד.] מצב בטוח הוא מצב שממנו יש סכנה שנגיע ל-deadlock, ואפשר להתחמק מכך רק על ידי שימוש בסידור מסוים של התהליכים.

שאלה 6

בחרו את הפעולה היקרה ביותר במערכת קבצים מבוססת I-Node במונחים של מעברי בלוקים של הדיסק (disk block transfers) בהנחה שלא קיימים נתונים רלוונטיים בזיכרון המטמון (buffer cache) לפני הפעולות:

א. סדרת עשר קריאת של תו אחד באמצעות fgetc מקובץ קטן מאוד בגודל 10 Byte

ב. קריאת בלוק אחד באמצעות read מתחילתו של קובץ גדול מאוד בגודל 1 GigaByte

ג. קריאת בלוק מספר שלושים באמצעות read מתוך קובץ גדול מאוד בגודל 1 GigaByte ששמור ברצף בגישה אקראית(ישירה)

[✓ ד.] קריאת בלוק מספר שלושים באמצעות read מתוך קובץ גדול מאוד בגודל 1 GigaByte ששמור **לא!** ברצף בגישה אקראית(ישירה)

חלק ב (55 נקודות)

ענו על השאלות 7-9.

שאלה 7 (23 נקודות)

א. (17 נק')

אנא סיימו את התוכנית להלן עבור בעיית קוראים-כותבים(כאשר קוראים יכולים לקרוא במקביל, כותבים לא יכולים לכתוב במקביל לא אחד עם השני ולא עם הקוראים). **הפתרון צריך לתת העדפה לכותבים לגבי מצטרפים חדשים("חדש" זה מי שעדיין לא הספיק לבצע את השורה הראשונה של הקוד).** הניחו **שכל תוכניות המשתמש מושהות עד סיום ביצוע קריאת מערכת.** הסמפורים הם אובייקטים של מערכת(כרגיל).

מלאו בכל שורה ריקה פעולה אחת בלבד(פקודה רגילה אחת או פעולה אחת על אמצעי סנכרון).

- ♦ semaphore BlockRead = 1; // binary semaphore used to block readers
- ♦ semaphore BlockWrite = 1; //binary semaphore used to allow ONE writer or MANY readers
- ♦ int Readers = 0, Writers = 0; // count of readers and writers in critical section
- ♦ semaphore RMutex = 1; // binary semaphore for Readers
- ♦ semaphore WMutex = 1; // binary semaphore for Writers

Reader

העדיפות לכותבים מוסדרת BlockRead

```
1.  down(BlockRead)          ; // at most one reader can go before a pending write
2.  down(RMutex)             ;
3.  Readers++;
4.  if (Readers == 1) // first reader acquire write lock
5.      down(BlockWrite)     ;
6.  up(RMutex)               ;
7.  up(BlockRead)            ;
8.  < Do the Reading >
9.  down(RMutex)             ;
10. Readers--;
11. if (Readers == 0) // last (only) reader releases write lock
12.     up(BlockWrite)       ;
13. up(RMutex)               ;
```

Writer

```
1. down(WMutex)              ;
2. Writers++;
3. if (Writers == 1)
```

נא לא לכתוב בשוליים

```
4.  down(BlockRead) ; // block readers
5.  up(WMutex) ;
6.  down(BlockWrite) ;
7.  < Do the Writing >
8.  up(BlockWrite) ;
9.  down(WMutex) ;
10. Writers--;
11. if (Writers == 0)
12. up(BlockRead) ; // enable readers
13. up(WMutex) ;
```

הסבירו כיצד הפרוטוקול עובד: האם בכלל ואיך מבטיח מניעה הדדית, התקדמות(העדר קיפאון) והמתנה מוגבלת.

מניעה הדדית: מובטחת ע"י הסמפור BlockWrite. אם יש כותב בקטע הקריטי, ע"פ הפרוטוקול כותבים אחרים ממתינים, וכן קוראים חדשים ומעוניינים להיכנס צריכים להמתין על הסמפור BlockWrite (אם יש כותב בקטע הקריטי, לא בוצע ~~שם~~ up לBlockWrite ולכן ש-Readers עשו תמיד המתנה).

התקדמות: אין קיפאון משום שלא מתקיים תנאי המעגליות, שהוא תנאי הכרחי לקיפאון. ~~[illegible]~~ המשאבים הרלוונטיים לקיפאון הם BlockRead וBlockWrite, כותבים וקוראים ממתינים למשאבים באותו הסדר - קודם BlockRead, ולאחר מכן BlockWrite (אין המתנה של כותבים לWMutex עבור כותב אחר שממתין לRead, ~~[illegible]~~ ובהמתנה עבור RMutex ו-Write).

(~~[illegible]~~: המתנה מוגבלת: אם כותב ממתין לקוראים (על BlockReads) - קוראים חדשים לא ייכנסו לקטע הקריטי ולאחר זמן סופי הקוראים יסיימו והכותב ייכנס. אם כותב ממתין לכותבים, לאחר זמן סופי נשחרר BlockWrite.

ב. (6 נק')

הסבירו מה, באיזה מידה ואיך במנגנון הסנכרון מבטיח העדפת הכותבים.

תשובה והסבר:

~~[illegible]~~ אם ישנם כותבים וממתינים על BlockWrite, אז ערך הסמפור BlockRead (ש/ו) 0 ואז קוראים חדשים ימתינו בשורה 1 ולא יוכלו "לעקוף" את הכותבים.

~~[illegible]~~ לעומת זאת, אם קוראים ממתינים על BlockRead ויש כותבים בקטע הקריטי, כותבים חדשים לא יצטרכו להמתין על BlockRead (כי Writers>1) וכך "יעקפו" את הקוראים הממתינים.

העדיפות מוגבלת בכך שאם קורא תפס את BlockRead, כותבים חדשים "אלצו" להמתין על BlockRead עד שהכותב ישחרר אותו. במצב כזה הקוראים יתפסו את BlockWrite ויעכבו את ~~[illegible]~~ BlockRead, ולא הכותבים יעברו להמתין על BlockWrite, ע"פ ההסבר לעיל.

23
(7)

נא לא לכתוב בשוליים

שאלה 8 (18 נקודות)

Page Table Entry

נניח שיש לנו טבלת דפים בשתי רמות. אורך כל כניסת טבלת הדפים(PTE) הוא 4 בתים.

נניח שאנו רוצים שכל טבלת דפים בודדת(ראשית וכל אחת ממשניות) תתפוס בדיוק דף זיכרון אחד.

א (11 נק')

מה צריך להיות הפורמט החלוקה לסיביות של כתובת וירטואלית באורך 32 סיביות שמתאים לזה?

הערה: למרות שלא ניתן גודל הדף, ניתן לחשב אותו ממשוואה שתאפשר את זה או בניסוי וטעיה.

תשובה, חישוב והסבר

החלוקה היא: 10 סיביות לאינדקס בטבלה הראשית, 10 סיביות לאינדקס בטבלה משנית, ו-12 סיביות offset.

הסבר: אם גודל הדף הוא 2^n bytes וגודל שדה אינדקס בשני הוא m סיביות, אז גודל הטבלה המשנית יהיה 2^m שורות X 4 בתים בשורה, כלומר 2^{m+2} bytes. בהשוואה ל-2^n נקבל m=n-2. כאשר, ~~[illegible]~~ $34-2n$ גודל שדה אינדקס ראשי הוא 32-(n-2)-n כלומר 34-2n, ומחשוב דומה גודל טבלה ראשית הוא 2^{36-2n} bytes, ומהשוואה ל-2^n בתים נקבל 36-2n=n, כלומר n=12.

ב. (4 נק')

כמה דפים בסה"כ יתפסו כל טבלאות הדפים ביחד(ראשית ומשניות) בהנחה שכל תהליך מקבל את מרחב הזיכרון המרבי האפשרי לפי נתוני השאלה?

הסבר וחישוב:

הטבלה הראשית תתפוס דף. ע"פ הנחת, שדה האינדקס הראשי הוא בגודל 10 סיביות, כלומר יש 2^{10} טבלאות משניות, כל אחת בגודל דף. סה"כ נקבל 1025 דפים בגודל 2^{12} = 4KB, כלומר 4100KB שהם קצת יותר מ-4MB. (4MB=4096KB).

ג. (3 נק')

מה היתרון בשימוש בטבלת הדפים במספר רמות יחסית לטבלת הדפים ברמה אחת(שטוחה)?

אין חיפוש- אינדקס

15
(8)

תשובה:

היתרון בשימוש בטבלת דפים במספר רמות הוא שבחיפוש דף נצטרך לחפש בדפים בודדים בלבד, כמספר הרמות, לעומת מספר רב יותר של דפים. למשל, ~~[illegible]~~ עבור דפים בגודל זהה לתחילת השאלה ומרחב זיכרון זהה, נקבל כי טבלת הדפים שלה 2^{20} שורות, תשתרע על פני 2^{22} בתים שהם 2^{10} דפים, פי $512=2^9$ (!) ~~[illegible]~~ משני הדפים שנדרשו בחיפוש בשתי רמות.

שאלון 511

92.52.5 M2

שאלה 9 (14 נקודות)

נתונה מערכת קבצים בדיסק קשיח דמוית ext2 מבוססת I –NODES עם 10 מצביעים ישירים, והשאר המצביעים הלא ישירים הרגילים ברמות שונות. במערכת זו מטמון מערכת קבצים מבוטל. מכני וטבלת ה I-NODES מוחזקים בתחילת הדיסק.

גודל הבלוק הוא 4 KB , אורך הכתובת 8 BYTE .

א. (7 נק')

התוכנית מגרילה בכל פעם באופן רנדומאלי אה המיקום(offset) בקובץ נתון וקוראת תו בודד ממיקום זה בקובץ בעזרת הפונקציה:

```
int read(FILE*fp, long offset) {
        fseek(fp, offset, SEEK_SET);
        return fgetc(fp);
}
```

גישה אקראית לקובץ

לאחר הרצת התכנית מספר רב של פעמים על קבצים שונים שמו שמשך הזמן הנדרש להרצת התכנית שונה:

עבור קבצים קטנים מ 40 KB משך ההרצה קצר בצורה משמעותית מקבצים גדולים מ 2 GB . כיצד ניתן להסביר תופעה זו בהסתמך על מאפייני המערכת?

הסבירו בקצרה אך במדויק.

הסבר וחישוב:

קבצים קטנים, בגודל עד 40KB, ניתנים לשמירה בעזרת 10 בלוקי

הdirect (10×4KB=40KB), לכן גישה אקראית אליהם תדרוש קריאת בלוק

אחד בלבד (פר) לi-node). ~~[illegible]~~ בגישה.

ה indirect ניתן לשמור עד $\frac{4KB}{8B}=512$ מצביעים לבלוקים

שהם 2048KB נוספים, ובשכבה d-indirect ניתן לשמור עד $512^2=2^{18}$

מצביעים לבלוקים שהם 2^{30}byte כלומר 1GB נוספים, לכן בקבצים

גדולים (2GB) יש להשתמש שמוש בשכבה triple indirect, לגישה אקראית

לקובץ נדרש לבצע קריאה של עד שלושה בלוקים נוספים ביחס לקבצים קטנים,

דבר המאט משמעותית את הפעולה.

ב. (3 נק')

האם התופעה הנזכרת לעיל בהכרח תהיה קיימת במערכת שמחזיקה את הקבצים לפי הגודל שלהם בדרך הבאה: הקבצים הגדולים בסוף הדיסק הקשיח והקטנים בתחילתו?

אין צורך להתייחס לשאלה איך היא עושה את זה.

תשובה והסבר:

כן. ע"פ הנאמר בתחילת השאלה, קבצי הi-node מצויים בתחילת

הדיסק, ולכן קריאות מידע של קבצים גדולים, הנמצאים בסוף הדיסק, ו

תגרור seek time ~~[illegible]~~ גדול יותר לבלוקי המידע של הקבצים

הגדולים, כמו בנוסף לכך שצריך לקרוא בלוקים נוספים

והמצביעים single, double ו- triple indirect .

ג. (4 נק')

תוכנית חיפוש תבניות(רצפים של תווים) בקבצים במערכת הנ"ל קוראת את כל תוכן הקובץ למערך בזיכרון ראשי. מהו מספר הבלוקים השונים המקסימאלי שצריך לקרוא מתוך מערכת הקבצים הנ"ל לזיכרון על מנת לבצע חיפוש בקובץ שגודלו 2048 KB?

14
(9)

חישוב והסבר :

קובץ שגודלו 2048KB = 512 blocks, ~~[illegible]~~ 10 מתוך
ומצביעים ל 512 הבלוקים שלו שמופיעים בשדות ה direct,
ו 502 נוספים שמורים בשדה ה(indirect),
לכן יש לקרוא בלוק אחד ל i-node, בלוק מצביעים של
שדה ה indirect, ועוד 512 בלוקי מידע, סה"כ 514 בלוקים.
(כולל i-node).

המשך הבחינה בעמוד הבא

חלק ג (15 נקודות)

ענו על שלוש השאלות 10-12.

(5 נקודות לכל שאלה; 15 נקודות לחלק זה).

5
(10)

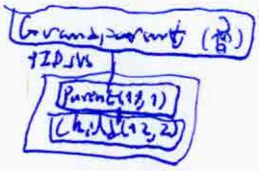
שאלה 10

הסבירו את ההבדל בין המושגים: פונקציות ספריה(printf למשל), ממשק(interface) קריאת מערכת(write למשל) וקריאת מערכת עצמה TRAP. ציינו בשביל מה כל אחד מהם משמש.

תשובה והסבר

קריאות TRAP מעבירה את השליטה לקרנל המערכת, על מנת שיוכל לבצע הוראות מיוחסות (כמו כתיבה לדיסק).
קריאות מערכת מעבירה את הפרמטרים הרלוונטיים לפונקציות ומתאימים ואז מבצעת TRAP ע"מ שהקרנל ידע מה לעשות -ואז מבצעת את ההוראה במצב מיוחס.
(מעטפת למשל כתיבת buffer לwrite). הן הפונקציות הספרייה כמו printf מספקות ממשק לקריאות המערכת ומקלות את השימוש בהן, למשל printf שתומכת formatted output ~~[illegible]~~ כמו בעזרת רצף התווים [illegible]

שאלה 11



מה יהיה הפלט אחרי הרצת הפקודות הבאות במערכת XV6 שתומכת בקונטיינרים?
ידוע שה PID הגדול ביותר שהיה במערכת לפני הרצת unshare (PID_NS) היה 10.
ניתן להניח שכל קריאות מערכת מצליחות.

```
unshare(PID_NS);
pid = fork();  //pid = 11 / 1?
if (pid > 0)  printf(stdout, "Grandparent's perspective on the child. PID=%d \n", pid);
else {
   pid = fork();
   if (pid > 0) printf (stdout, "Parent's perspective on his child:  PID=%d \n", pid);
   else printf (stdout, " Child's perspective on his parent:  PID=%d \n", getppid());
   }
```

5
(11)

תשובה והסבר:

בוצע unshare ולכן fork, אך נוצר pid namespace חדש, עבור הנכד והבן.
~~[illegible]~~ רואה את בנו כ11 ב rootnamespace, ~~[illegible]~~ (הנכד) רואה עצמו ואת בנו כחלק מה-namespace החדש, והבן רואה עצמו ואת אביו ב NS החדש.

Grandparent's perspective ... = 11
Parent's perspective :.. = 2
Child's perspective... = 1

5
(12)

שאלה 12

הסבירו היטב את ההבדל בין VIRTUALIZATION ל CONTAINERIZATION . התייחסו במיוחד להבדלים באופן המימוש ולכמות המשאבים הדרושים.

תשובה והסבר :

לוירטואליזציה מספר רמות. אפשר לממש וירטואליזציה בInstruction Sets
(כמו B הוראות מכונה מתורגמות) או בחומרה (כמו ניתן להריץ, למשל,
מע' הפעלה אורחת עם גרעין שונה מאשר אמיתי למשל עם Containers).
בשימוש בקונטיינרים (וירטואליזציה ברמת ~~[illegible]~~ האפליקציה) דרוש
פחות משאבים, משום שהשימוש של הגרעין בידוד בין הקבוצות
של תהליכים והתהליכים עצמם רצים עם אותו גרעין מע' ההפעלה.

בהצלחה!